МЕЖДУНАРОДНЫЙ
ВОСТОЧНО-ЕВРОПЕЙСКИЙ
КОЛЛЕДЖ

ПРОГРАММИРОВАНИЕ ИНТЕРФЕЙСОВ • JAVASCRIPT • DOM

Проектирование и разработка интерфейсов пользователя

Преподаватель

Коляда Никита Владимирович

Тема 4

МЕЖДУНАРОДНЫЙ ВОСТОЧНО-ЕВРОПЕЙСКИЙ КОЛЛЕДЖ

ПЛАН ЗАНЯТИЯ

1. Основы JavaScript
Роль языка, переменные const/let, типы данных и подключение defer.

2. Манипуляция DOM
Дерево элементов, выборка через querySelector и свойства узлов.

3. События и классы
Слушатель addEventListener, объект события, forEach и classList.

4. Практика: навигация и тема
Подсветка активной ссылки меню и переключатель темной темы.

5. Итоги занятия и разбор типичных ошибок
Разбор типичных ошибок верстки, методы отладки и закрепление материала

РОЛЬ JAVASCRIPT

Три столпа веб-разработки

```
<!-- 1. HTML: Структура и смысл -->
<button id="theme-btn">Сменить тему</button>

/* 2. CSS: Внешний вид и анимация */
#theme-btn { background: #3b82f6; color: #fff; }
.dark-theme { background: #181a1f; }

// 3. JavaScript: Логика и интерактивность
const btn = document.querySelector('#theme-btn');
btn.addEventListener('click', () => {
  document.body.classList.toggle('dark-theme');
});
```

ДИНАМИЧЕСКИЙ ВЕБ-ИНТЕРФЕЙС

- **Язык сценариев браузера**
  JavaScript выполняется прямо на компьютере пользователя внутри движка браузера.
- **Реакция на действия человека**
  Слушает клики мыши, нажатия клавиш, прокрутку страницы и ввод текста.
- **Изменение страницы на лету**
  Обновляет текст, меняет классы и переключает темы оформления без перезагрузки.
- **Стандарт ECMAScript (ES6+)**
  Современный синтаксис гарантирует единообразную работу во всех браузерах.

Как JavaScript дополняет связку HTML и CSS в браузере?

index.html (подключение <script>)

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <link rel="stylesheet" href="css/style.css">
  <!--  ИДЕАЛЬНО: defer загружает в фоне и ждет DOM -->
  <script src="js/main.js" defer></script>
</head>
<body>
  <!-- Видимый контент страницы -->
</body>
```

АТРИБУТ DEFER И ПАРСИНГ HTML

- **Проблема синхронных скриптов**
  Обычный <script> останавливает парсинг HTML до полной загрузки файла кода.
- **Фоновое скачивание с defer**
  Браузер скачивает JS параллельно с верстской, не задерживая показ интерфейса.
- **Выполнение строго после DOM**
  Скрипт с defer гарантированно запускается, когда все теги уже созданы в памяти.
- **Порядок выполнения скриптов**
  Несколько файлов с defer выполняются строго в том порядке, в каком они подключены.

Почему скрипты в секции <head> обязательно подключать с атрибутом defer?

js/main.js (объявление переменных)

```
// 1. const — неизменяемая ссылка (по умолчанию)
const MAX_USERS = 100;
const header = document.querySelector('.header');

// 2. let — переменная, значение которой меняется
let activeCount = 0;
activeCount = activeCount + 1; // Значение обновлено

//  3. var — устаревший синтаксис (функциональная область)
var oldVariable = 'не использовать';

// Правило именования: camelCase
const activeNavLink = 'home';
```

КОНТРОЛЬ ПАМЯТИ И ОБЛАСТЬ ВИДИМОСТИ

- **Приоритет const в 95% случаев**
  Защищает переменную от случайного переопределения в коде программы.
- **Использование let для счетчиков**
  Применяется только тогда, когда значение переменной действительно перезаписывается.
- **Блочная область видимости { }**
  let и const живут строго внутри своего блока кода (if, for, функция), не засоряя память.
- **Стиль именования camelCase**
  Общепринятый мировой стандарт в JS: первое слово со строчной, следующие с заглавной.

Почему современный стандарт запрещает использование устаревшего var?

js/main.js (типы и typeof)

```
// 1. String (Текст): кавычки '', "", или шаблоны ``
const title = 'Каталог рецептов';

// 2. Number (Числа): целые и дробные
const price = 450;
const rating = 4.8;

// 3. Boolean (Булев тип): true или false
const isOpen = true;
const isClosed = false;

// 4. Проверка типа оператором typeof
console.log(typeof title);  // 'string'
console.log(typeof price);  // 'number'
console.log(typeof isOpen); // 'boolean'
```

ПРИМИТИВЫ И ДИНАМИЧЕСКАЯ ТИПИЗАЦИЯ

- **Динамическая типизация**
  Тип переменной определяется автоматически по значению, лежащему в ней прямо сейчас.
- **Строки String**
  Текстовые данные для заголовков, имен классов, путей к файлам и сообщений.
- **Числа Number**
  Единый тип для целых и дробных чисел, расчетов времени, счетчиков и координат.
- **Логический тип Boolean**
  Флаги состояний: открыто ли меню, темная ли тема, активен ли пункт.

Какие базовые примитивные типы данных существуют в JavaScript?

СХЕМА: ДЕРЕВО DOM

1. WINDOW И DOCUMENT
Окружение и корень документа

```
window.document
```

Точка входа в документ:

window представляет окно браузера, а document — загруженную HTML-страницу и точку старта для поиска любых тегов.

2. ВЕТВИ И УЗЛЫ (NODES)
Иерархия тегов страницы

```
document.documentElement
```

Древовидная связь тегов:

Каждый HTML-тег становится объектом в дереве DOM. Родители хранят детей: html -> head + body -> header, main, nav.

3. УПРАВЛЕНИЕ ИЗ JS
Манипуляция через DOM API

```
el.classList.toggle('active')
```

Оживление страницы:

JavaScript находит узлы через querySelector, слушает клики пользователя и меняет стили и классы без перезагрузки.

Как браузер представляет HTML-документ в виде дерева объектов?

```
js/main.js (DOM API)

// document — глобальная точка входа в DOM-дерево
console.log(document.title); // Чтение заголовка вкладки
console.log(document.body);  // Ссылка на узел <body>

// Поиск по селекторам CSS:
const mainHeader = document.querySelector('.site-header');

// Каждый элемент страницы — это объект со свойствами:
console.log(mainHeader.tagName);   // 'HEADER'
console.log(mainHeader.className); // 'site-header'
```

ДОСТУП К ЭЛЕМЕНТАМ СТРАНИЦЫ

- **DOM = Document Object Model**
  Интерфейс программирования, представляющий каждый HTML-тег как JS-объект.
- **Глобальный объект document**
  Предоставляет методы поиска, создания, изменения и удаления узлов страницы.
- **Связь HTML и JS в реальном времени**
  Изменение свойства объекта DOM немедленно отражается на экране браузера.
- **Узлы элементов (Element Nodes)**
  Обладают методами для работы с классами, стилями, атрибутами и событиями.

Что такое Document Object Model и как через него получать доступ к тегам?

js/main.js (селекторы DOM)

```
// 1. querySelector — возвращает ПЕРВЫЙ найденный элемент (или null)
const firstCard = document.querySelector('.recipe-card');
const brandLogo = document.querySelector('#logo');

// 2. querySelectorAll — возвращает ВСЕ элементы (NodeList)
const allNavLinks = document.querySelectorAll('.nav-link');
console.log(allNavLinks.length); // Количество найденных ссылок

// Принимаются абсолютно любые CSS-селекторы:
const activeBtn = document.querySelector('nav > ul li.active a');
```

МОЩЬ СОВРЕМЕННЫХ СЕЛЕКТОРОВ

- **Знакомый синтаксис CSS**
  Методы принимают классы .card, ID #header, атрибуты [data-id] и вложенность nav > a.
- **querySelector для одного узла**
  Идеален для поиска уникальных кнопок, форм, модалок или первого элемента.
- **querySelectorAll для коллекций**
  Возвращает псевдомассив NodeList со всеми совпавшими элементами страницы.
- **Безопасная проверка на null**
  Если элемент не найден, возвращается null, что легко проверить условием if.

В чем принципиальная разница между querySelector и querySelectorAll?

js/main.js (свойства DOM)

```
const heading = document.querySelector('h1');

// Чтение и безопасная запись текста
console.log(heading.textContent);
heading.textContent = 'Свежие рецепты недели';

// Прямое управление inline-стилями (через объект style)
heading.style.color = '#e11d48';
heading.style.fontSize = '32px'; // camelCase вместо font-size

// innerHTML парсит HTML (осторожно с пользовательским вводом!)
heading.innerHTML = '<span>Новые</span> Рецепты';
```

РАБОТА С СОДЕРЖИМЫМ И СТИЛЯМИ

- **textContent против XSS-уязвимостей**
  Вставляет текст как обычные символы, исключая внедрение вредоносных тегов.
- **Свойства style в camelCase**
  CSS-дефисы превращаются в заглавные буквы: background-color -> backgroundColor.
- **Inline-приоритет стилей**
  Стили, назначенные через el.style, имеют наивысший приоритет над файлом CSS.
- **Чтение исходного текста**
  textContent позволяет извлечь чистый текст внутри элемента без разметки.

Почему для изменения текста безопаснее использовать textContent, а не innerHTML?

js/main.js (методы classList)

```
const navMenu = document.querySelector('.nav-menu');

// 1. add — добавить класс (если уже есть — не дублирует)
navMenu.classList.add('is-open');

// 2. remove — удалить указанный класс
navMenu.classList.remove('hidden');

// 3. toggle — переключить класс (удаляет если был, добавляет если не было)
navMenu.classList.toggle('active');

// 4. contains — проверить наличие класса (true / false)
if (navMenu.classList.contains('active')) {
  console.log('Меню сейчас активно!');
}
```

БЕЗОПАСНАЯ РАБОТА С КЛАССАМИ

- **Интерфейс DOMTokenList**
  classList управляет списком классов как коллекцией, не затирая соседние классы.
- **Магия метода toggle()**
  Заменяет громоздкие проверки if/else: включает класс по клику и выключает повторным.
- **Метод contains() для условий**
  Позволяет проверять текущее состояние интерфейса (открыто ли меню, скрыта ли карточка).
- **Золотое правило верстки**
  Вся стилизация должна оставаться в CSS-классах, а JS лишь переключает эти классы.

Почему classList надежнее, чем прямая запись свойства className?

js/main.js (безопасный старт)

```
// Безопасная точка старта любого скрипта интерфейса
document.addEventListener('DOMContentLoaded', () => {
  console.log('HTML полностью распарсен, DOM готов!');

  // Безопасная инициализация компонентов:
  const navLinks = document.querySelectorAll('.nav-link');
  const themeBtn = document.querySelector('#theme-toggle');

  console.log(`Найдено ссылок меню: ${navLinks.length}`);
});
```

ГАРАНТИЯ ГОТОВНОСТИ ДЕРЕВА

- **Событие DOMContentLoaded**
  Срабатывает ровно тогда, когда браузер завершил построение дерева DOM из HTML.
- **Защита от ошибки null**
  Гарантирует, что селекторы querySelector найдут все элементы на странице.
- **Не ждет тяжелые картинки**
  В отличие от window.onload, не дожидается загрузки баннеров и видео, стартуя мгновенно.
- **Индустриальный стандарт**
  Оборачивание кода в DOMContentLoaded — обязательное правило чистого клиентского кода.

Какую гарантию дает событие DOMContentLoaded для запуска скрипта?

js/main.js (слушатель событий)

```
const btn = document.querySelector('.cta-button');

// Подписка на событие клика мыши
btn.addEventListener('click', (event) => {
  console.log('Кнопка нажата!');
  console.log('Целевой элемент:', event.target);
  console.log('Координата X:', event.clientX);
});

// На один элемент можно повесить несколько независимых
обработчиков:
btn.addEventListener('click', () => sendAnalytics());
```

МЕХАНИКА СЛУШАТЕЛЕЙ

- **Метод addEventListener**
  Универсальный стандарт подписки на события мыши, клавиатуры и браузера.
- **Объект события (Event)**
  Автоматически передается в функцию, сообщая координаты клика и нажатые клавиши.
- **Свойство event.target**
  Указывает точный исходный элемент, на котором физически произошел клик.
- **Множественные подписчики**
  Позволяет навешивать несколько независимых функций на одно и то же действие.

Как устроен метод addEventListener и какую информацию хранит объект event?

js/main.js (перебор NodeList)

```
const navLinks = document.querySelectorAll('.nav-link');

// Массовое добавление обработчика на каждую ссылку
navLinks.forEach((link, index) => {
  link.addEventListener('click', (e) => {
    // 1. Снять класс active у всех ссылок
    navLinks.forEach(l => l.classList.remove('active'));
    // 2. Добавить класс active кликнутой ссылке
    link.classList.add('active');
  });
});
```

МАССОВАЯ ОБРАБОТКА ЭЛЕМЕНТОВ

- **Метод forEach на NodeList**
  Последовательно вызывает функцию для каждого найденного элемента в коллекции.
- **Параметры колбэка (item, index)**
  В функцию передается сам DOM-узел и его порядковый номер в списке.
- **Компактные стрелочные функции**
  Синтаксис () => { } делает код перебора лаконичным и выразительным.
- **Паттерн переключения активного пункта**
  Очистить класс у всех элементов списка, затем включить у нажатого.

Как метод forEach позволяет массово оживить все пункты меню?

js/main.js (условия и сравнения)

```
const currentPath = window.location.pathname;

// Строгое сравнение (===) проверяет значение и тип
if (currentPath === '/recipes.html') {
  console.log('Открыт каталог рецептов');
} else if (currentPath === '/about.html') {
  console.log('Открыта страница о нас');
} else {
  console.log('Главная страница блога');
}

// Всегда использовать === вместо нестрогого ==
console.log(5 === '5'); // false (разные типы)
```

ВЕТВЛЕНИЕ И ПРОВЕРКИ

- **Конструкция if / else if / else**
  Выполняет нужный блок инструкций в зависимости от истинности условия.
- **Строгое равенство ===**
  Проверяет и совпадение значения, и тип данных, исключая скрытые баги JS.
- **Логические операторы && и ||**
  Позволяют объединять несколько условий в одну составную проверку.
- **Проверка текущего URL**
  Чтение window.location.pathname позволяет автоматически подсвечивать текущую страницу.

Как ветвление if / else управляет логикой веб-приложения?

СХЕМА: ПОТОК ОБРАБОТКИ СОБЫТИЯ

1. ДЕЙСТВИЕ (ACTION)

Пользователь совершает клик

```
button.addEventListener('click', ...)
```

Пользователь нажимает на кнопку переключения темы или пункт меню. Браузер регистрирует физическое событие клика.

2. ОБРАБОТЧИК (HANDLER)

Вызов JS-функции

```
(event) => { document.body.toggle }
```

Браузер создает объект события (event) и передает его в функцию-обработчик. Выполняется заложенная логика изменения классов.

3. ОБНОВЛЕНИЕ DOM

Мгновенная перерисовка

```
body.classList.add('dark-theme')
```

DOM мутирует: добавляется класс, меняются переменные CSS. Движок браузера моментально перерисовывает экран за доли миллисекунды.

Какую цепочку шагов проходит браузер от клика мыши до смены стилей?

ПРАКТИКА: ШАГ 1

index.html & js/main.js

```
<!-- В секции <head> index.html: -->
<script src="js/main.js" defer></script>

// В файле js/main.js:
console.log(' Скрипт веб-приложения успешно запущен!');

document.addEventListener('DOMContentLoaded', () => {
  console.log(' DOM-дерево готово к интерактивности');
});
```

ИНИЦИАЛИЗАЦИЯ И ПРОВЕРКА

- **Создание файла js/main.js**
  Размещаем скрипт в отдельной подпапке js согласно принятой архитектуре проекта.
- **Подключение с defer**
  Гарантирует загрузку скрипта без блокировки отрисовки стилей и интерфейса.
- **Диагностика через console.log**
  Вывод приветственного сообщения в консоль DevTools подтверждает запуск скрипта.
- **Готовность к манипуляциям**
  Оборачивание кода в DOMContentLoaded исключает ошибки обращения к тегам.

Как подготовить проект и убедиться в успешном подключении скрипта?

js/main.js (выборка пунктов)

```
document.addEventListener('DOMContentLoaded', () => {
  // Находим все ссылки внутри навигации
  const navLinks = document.querySelectorAll('.nav-link');

  console.log(`Найдено ссылок навигации: ${navLinks.length}`);

  // Проверяем адреса каждой ссылки
  navLinks.forEach((link) => {
    console.log('Пункт:', link.textContent.trim(), '->',
link.getAttribute('href'));
  });
});
```

ИНСПЕКЦИЯ КОЛЛЕКЦИИ ССЫЛОК

- **Выборка по общему классу**
  querySelectorAll('.nav-link') находит все пункты навигации в шапке.
- **Длина коллекции .length**
  Позволяет убедиться, что найдены все 3 или 4 запланированные ссылки меню.
- **Чтение атрибута href**
  Метод getAttribute('href') извлекает относительный адрес страницы для перехода.
- **Очистка пробелов trim()**
  Убирает лишние отступы и переносы строк вокруг текста внутри тега ссылки.

Как выбрать ссылки навигации и вывести их адреса в консоль?

js/main.js (подсветка меню)

```
const navLinks = document.querySelectorAll('.nav-link');

// Автоподсветка пункта по адресу текущей страницы
const currentFile = window.location.pathname.split('/').pop() ||
'index.html';

navLinks.forEach(link => {
  const linkHref = link.getAttribute('href');
  if (linkHref === currentFile) {
    link.classList.add('active');
  }
});
```

АВТООПРЕДЕЛЕНИЕ ТЕКУЩЕЙ СТРАНИЦЫ

- **Чтение имени открытого файла**
  Разбиение pathname извлекает имя текущей страницы: index.html или recipes.html.
- **Сравнение с адресом ссылки**
  Скрипт проверяет href каждой ссылки навигации на совпадение с текущей страницей.
- **Добавление класса active**
  Ссылка текущей страницы автоматически окрашивается в акцентный цвет CSS.
- **Универсальная работа**
  Один и тот же JS-код одинаково работает на всех страницах веб-сайта.

Как подсветить пункт меню текущей страницы или при клике?

ПРАКТИКА: ШАГ 4

js/main.js & css/style.css

```
/* В CSS: оформление темной темы */
body.dark-theme {
  background-color: #181a1f;
  color: #ffffff;
}

// В JavaScript: кнопка переключения темы
const themeBtn = document.querySelector('#theme-toggle');

themeBtn.addEventListener('click', () => {
  // Элегантное переключение класса в одну строчку!
  document.body.classList.toggle('dark-theme');
});
```

ЛОГИКА ПЕРЕКЛЮЧЕНИЯ ТЕМЫ

- **Умный метод toggle**
  Если класс dark-theme есть на body — он удаляется, если нет — добавляется.
- **Делегирование стилей в CSS**
  JavaScript не трогает цвета напрямую: переключение одного класса меняет весь стиль.
- **Плавность перехода в CSS**
  Добавление transition: background-color 0.3s делает смену темы плавной и комфортной.
- **Мгновенный отклик**
  Интерфейс мгновенно реагирует на клик пользователя без перезагрузки вкладки.

Как реализовать переключение темной темы одной строчкой через classList.toggle?

ПРАКТИКА: ШАГ 5

js/main.js (бургер-меню)

```
const burgerBtn = document.querySelector('.burger-btn');
const mobileMenu = document.querySelector('.mobile-menu');

burgerBtn.addEventListener('click', () => {
  // Переключаем видимость выпадающего меню
  mobileMenu.classList.toggle('is-open');

  // Анимируем иконку крестика на кнопке
  burgerBtn.classList.toggle('active');
});
```

ИНТЕРАКТИВНОЕ МОБИЛЬНОЕ МЕНЮ

- **Связка кнопки и шторки меню**
  По клику на кнопку бургер переключается класс .is-open у блока навигации.
- **Стили скрытия в CSS**
  По умолчанию меню имеет display: none или opacity: 0, а .is-open показывает его.
- **Анимация крестика**
  Включение класса .active на кнопке трансформирует три полоски в крестик закрытия.
- **UX адаптивного сайта**
  На смартфонах меню прячется за компактной кнопкой, экономя драгоценное место.

Как реализовать открытие и закрытие мобильного бургер-меню?

ОБОБЩЕНИЕ

Архитектура взаимодействия

```
/* Формула современного веб-интерфейса: */
/* 1. ПОЛЬЗОВАТЕЛЬ совершает действие (клик, ввод, скролл) */
/* 2. JAVASCRIPT перехватывает событие (addEventListener) */
/* 3. JAVASCRIPT находит элемент через селекторы (querySelector) */
/* 4. JAVASCRIPT меняет состояние в DOM (classList.toggle) */
/* 5. CSS мгновенно применяет новые визуальные стили */
/* 6. БРАУЗЕР перерисовывает экран за доли миллисекунды */
```

МОСТ МЕЖДУ СТРАНИЦЕЙ И ЧЕЛОВЕКОМ

- **JS — нервная система сайта**
  Связывает визуальные элементы с намерениями и кликами пользователя.
- **Минимум вмешательства в стили**
  JS переключает логические классы (.active, .is-open), оставляя оформление в CSS.
- **Нулевая задержка интерфейса**
  Все изменения происходят локально в браузере без обращения к серверу.
- **Фундамент для веб-приложений**
  Освоенный DOM API — прямой шаг к созданию каталогов, корзин и фильтров.

В чем заключается фундаментальная роль JavaScript в веб-приложении?

РАЗБОР ОШИБОК

4 ТИПИЧНЫЕ ОШИБКИ ПРИ РАБОТЕ С DOM И JAVASCRIPT

Скрипт запущен до построения DOM (null)

✕ **Ошибка:** Код выполняется раньше, чем браузер дочитал тег кнопки. querySelector возвращает null.

✓ **Решение:** Всегда добавлять атрибут defer или слушать событие DOMContentLoaded.

Опечатка в точке селектора класса

✕ **Ошибка:** Написано querySelector('nav-link') вместо '.nav-link'. Браузер ищет несуществующий тег.

✓ **Решение:** Помнить, что селектор класса обязательно начинается с точки ('.class').

querySelector вместо querySelectorAll

✕ **Ошибка:** Попытка вызвать forEach на результате querySelector приводит к фатальной ошибке TypeError.

✓ **Решение:** Для коллекций элементов ВСЕГДА использовать querySelectorAll.

Забытый classList при добавлении

✕ **Ошибка:** Попытка присвоить el.class = 'active' вместо el.classList.add('active').

✓ **Решение:** Для управления классами использовать исключительно свойство classList.

С какими распространенными ошибками при работе с DOM сталкиваются начинающие?

МЕЖДУНАРОДНЫЙ
ВОСТОЧНО-ЕВРОПЕЙСКИЙ
КОЛЛЕДЖ

ДО ВСТРЕЧИ!

УВИДИМСЯ НА СЛЕДУЮЩЕМ ЗАНЯТИИ